

OpenSeek-4 Conala Concat Strings 详细技术报告

1. 项目背景与任务概述

本技术报告基于现有的《OpenSeek-4 Conala Concat Strings 技术报告》和当前提交代码 `submit.py` 重新整理、扩展而成。原始报告已经给出了任务概述、数据集描述、核心函数说明和实验结果，但整体偏简略，缺少完整的数据流分析、模块职责划分、异常路径说明、输出文件结构、可复现流程，以及代码与报告之间的一致性检查。因此，本报告在原有内容基础上补充更细粒度的实现解释，并对当前代码中存在的命名不一致和回退求解器逻辑错误进行专门说明。

本任务对应 OpenSeek 系列中的第 4 个任务：**Conala Concat Strings**。任务目标是对输入中的字符串列表进行顺序拼接，并返回拼接后的字符串。输入通常是一个可以被 Python 字面量解析的字符串列表，例如：

```
1 ['p', 'that.', 'o']
```

对应输出为：

```
1 pthat.o
```

从算法角度看，该任务本身并不复杂，核心操作可以概括为：

```
1 strings = ast.literal_eval(input_text.strip())
2 return ''.join(strings)
```

但是当前代码并不是简单手写一个固定 solver，而是实现了一个“大模型代码生成 + 本地样例验证 + 错误反馈重试 + 回退求解器 + 测试集预测输出”的自动化流程。因此，本项目的重点不只是字符串拼接本身，还包括如何让模型根据任务定义和样例自动生成可执行 Python 代码，并通过训练样例验证其正确性，最后批量生成测试集提交文件。

2. 任务输入输出规范

2.1 输入格式

输入 `input_text` 是完整字符串形式的数据，内容本质上是 Python 列表字面量。列表元素均为字符串，列表长度不固定。典型输入如下：

```
1 ['hello', 'world']
2 ['p', 'that.', 'o']
3 ['fk', 'buttoned', 'earef', 'IasW']
```

由于输入是字符串形式，因此不能直接当作列表使用，需要先解析。代码中要求使用：

```
1 ast.literal_eval(input_text.strip())
```

该方法比 `eval` 更安全，只允许解析 Python 字面量结构，例如字符串、列表、字典、数字、布尔值等，适合处理此类任务输入。

2.2 输出格式

输出为一个字符串，表示将输入列表中的所有字符串元素按原顺序连接后的结果。输出中不额外添加空格、逗号、换行或列表符号。

例如：

输入	输出
<code>['p', 'that.', 'o']</code>	<code>pthat.o</code>
<code>['hello', 'world']</code>	<code>helloworld</code>
<code>['a', '', 'b']</code>	<code>ab</code>
<code>[]</code>	空字符串

2.3 样例与测试集的区别

数据文件中包含两类样本：

- 1. 训练/示例样本 **examples**：包含输入和标准输出，可用于提示词构造、代码验证和准确率计算。
- 2. 测试样本 **test_samples**：通常只包含输入，需要程序生成预测结果，并保存为 JSONL 提交文件。

当前代码会先在 `examples` 上验证生成的 `solver`，再对 `test_samples` 进行预测。

3. 数据集组织方式

当前代码通过 `load_task(file_path)` 读取 JSON 格式任务文件。该文件预计包含如下字段：

```
1 {
2   "task_id": "openseek-4",
3   "task_name": "Conala Concat Strings",
4   "Definition": [
5     "Given a list of strings, concatenate them in order."
6   ],
7   "examples": [
8     {
9       "id": "example-id",
10      "input": ["'p'", "'that.', 'o'"],
11      "output": ["pthat.o"]
12    }
13  ],
14  "test_samples": [
15    {
16      "id": "test-id",
17      "input": ["'fk'", "'buttoned', 'earef', 'IasW']"
```

```
18     }
19   ]
20 }
```

`load_task` 返回五项内容：

```
1 task_id, task_name, definition, examples, test_samples
```

各字段含义如下：

字段	类型	作用
<code>task_id</code>	<code>str</code>	任务编号，例如 <code>openseek-4</code>
<code>task_name</code>	<code>str</code>	任务名称，例如 <code>Conala Concat Strings</code>
<code>Definition</code>	<code>List[str]</code>	自然语言任务定义，用于构造提示词
<code>examples</code>	<code>List[Dict]</code>	带标签样例，用于 few-shot 提示和验证
<code>test_samples</code>	<code>List[Dict]</code>	无标签测试样本，用于最终预测

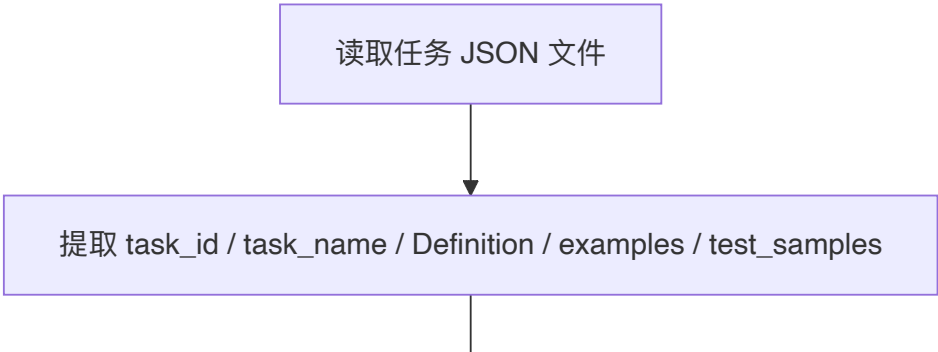
这种结构将任务元信息、样例数据和测试数据统一放入一个 JSON 文件中，使得主流程可以不依赖硬编码样例，而是根据数据文件动态构建提示词和评测流程。

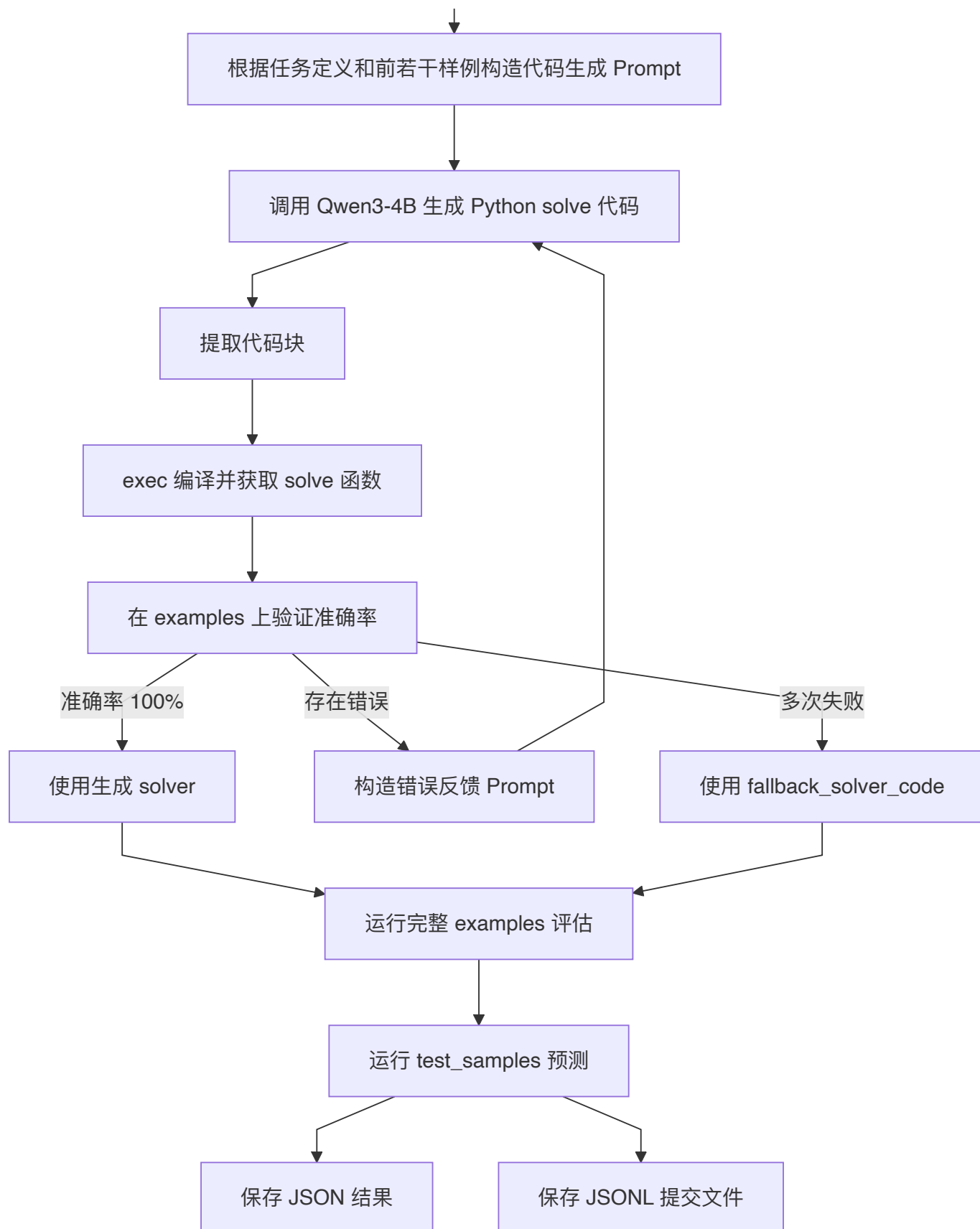
4. 代码整体架构

当前代码可以划分为七个核心模块：

1. **数据加载模块**：读取任务 JSON 文件，提取任务信息、样例和测试集。
2. **模型调用模块**：通过 OpenAI 兼容接口调用 Qwen3-4B 生成 Python 代码。
3. **代码提取模块**：从模型输出中提取 Markdown 代码块或纯代码文本。
4. **回退求解器模块**：当模型生成失败或验证不通过时使用保底 solver。
5. **代码编译模块**：使用 `exec` 动态执行生成代码，并提取 `solve` 函数。
6. **验证模块**：在 `examples` 上运行 solver，计算准确率并记录错误样例。
7. **预测与保存模块**：对 `examples` 和 `test_samples` 运行 solver，并保存 JSON 与 JSONL 文件。

整体流程如下：





该流程体现了一种“生成-验证-修正”的自动代码生成框架。模型生成的代码不会被直接用于测试集，而是先在带标签样例上进行本地验证；只有通过验证后，才会进入最终预测流程。

5. 核心模块详细分析

5.1 数据加载函数 `load_task`

函数定义如下：

```
1 def load_task(file_path: str) -> Tuple[str, str, List[str], List[Dict[str, Any]],  
  List[Dict[str, Any]]]:  
2     with open(file_path, 'r', encoding='utf-8') as f:  
3         data = json.load(f)  
4     return (  
5         data.get("task_id", ""),  
6         data.get("task_name", ""),  
7         data.get("Definition", []),  
8         data.get("examples", []),  
9         data.get("test_samples", []),  
10    )
```

该函数负责读取任务数据文件。实现上使用 `data.get(key, default)`，因此即使某个字段缺失，也不会直接抛出 `KeyError`，而是返回默认值。这种设计增强了程序对数据文件格式波动的容错能力。

不过，该函数目前只完成“读取”和“字段提取”，没有对字段类型和样本结构进行严格校验。例如：

- `examples` 是否为列表；
- 每个样例是否包含 `input`；
- 带标签样例是否包含 `output`；
- `output` 是否是列表，且至少包含一个标准答案；
- `test_samples` 是否为空。

如果数据文件格式不符合预期，错误可能会延迟到验证或预测阶段才暴露。为了提高工程健壮性，可以在该函数中增加数据校验逻辑。

建议增强版本如下：

```
1 def load_task(file_path: str):  
2     with open(file_path, 'r', encoding='utf-8') as f:  
3         data = json.load(f)  
4  
5     required_keys = ["task_id", "task_name", "Definition", "examples",  
6 "test_samples"]  
7     for key in required_keys:  
8         if key not in data:  
9             raise ValueError(f"任务文件缺少必要字段: {key}")  
10  
11     if not isinstance(data["examples"], list):  
12         raise TypeError("examples 必须是列表")  
13  
14     if not isinstance(data["test_samples"], list):  
15         raise TypeError("test_samples 必须是列表")  
16  
17     return (  
18         data["task_id"],
```

```

18         data["task_name"],
19         data["Definition"],
20         data["examples"],
21         data["test_samples"],
22     )

```

5.2 模型调用函数 `qwen_api`

代码中使用 OpenAI 兼容客户端调用 Qwen3-4B:

```

1 client = OpenAI(
2     api_key="dummy",
3     base_url=".../v1",
4 )

```

调用函数为:

```

1 def qwen_api(messages: List[Dict[str, str]], model: str = "/Qwen3-4B/Qwen/Qwen3-4B",
2   retries: int = 3) -> str:
3     for attempt in range(retries):
4         try:
5             res = client.chat.completions.create(
6                 model=model,
7                 messages=messages,
8                 temperature=0.0,
9             )
10            return res.choices[0].message.content or ""
11        except Exception as e:
12            if attempt == retries - 1:
13                print(f"\nAPI 调用失败: {e}")
14                return ""
15            time.sleep(2)
16    return ""

```

该函数的设计特点包括:

1. **固定低温度生成:** `temperature=0.0`，用于降低模型输出随机性，使生成代码更稳定。
2. **重试机制:** 默认最多重试 3 次，适合处理临时网络错误、服务端繁忙或接口异常。
3. **失败返回空字符串:** 最后一次失败后不抛出异常，而是返回空字符串，后续代码会尝试提取和编译，最终可能进入回退流程。

该设计适合比赛或批量运行场景，因为它不会因为一次 API 调用失败而立即中断整个程序。不过也存在不足:

- 没有记录每次失败的详细日志;
- 没有区分超时、限流、认证失败等错误类型;
- 没有设置 `max_tokens`，可能受到默认输出长度限制;
- API 地址硬编码在代码中，不利于迁移和提交。

更好的工程做法是通过环境变量或配置文件传入 API 参数:

```

1 client = OpenAI(
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),
3     base_url=os.getenv("OPENAI_BASE_URL"),
4 )

```

这样可以避免将服务地址或密钥相关信息写死在源码中。

5.3 代码块提取函数 `extract_code`

模型输出可能有两种形式：

第一种是 Markdown 代码块：

```

1 ```python
2 def solve(input_text: str) -> str:
3     ...
4 ```

```

第二种是纯代码文本：

```

1 def solve(input_text: str) -> str:
2     ...

```

因此代码使用正则表达式提取代码块：

```

1 CODE_BLOCK_PATTERN = re.compile(
2     r"```python\s*(.*?)```|```\s*(.*?)```",
3     re.DOTALL | re.IGNORECASE
4 )

```

该正则表达式支持两类代码块：

1. 显式标注 `python` 的代码块；
2. 未标注语言类型的普通代码块。

函数逻辑为：

```

1 def extract_code(text: str) -> str:
2     if not text:
3         return ""
4     match = CODE_BLOCK_PATTERN.search(text)
5     if match:
6         return (match.group(1) or match.group(2) or "").strip()
7     return text.strip()

```

如果匹配到代码块，则返回代码块内部内容；否则将整个模型输出作为代码返回。这种设计比较实用，因为不同模型在代码生成时的格式并不总是稳定。

但该方法也有局限：

- 如果模型输出多个代码块，只会取第一个；
- 如果代码块中包含解释文字和代码混排，可能提取不干净；
- 如果模型输出不是 Python 代码而是自然语言，后续 `exec` 会失败。

在当前流程中，这些问题会由 `compile_solver` 和 `validate_solver` 捕获，并触发下一轮修正或回退。

5.4 Prompt 构造函数 `build_code_generation_prompt`

当前代码通过任务名称、任务定义和前 10 个样例构造提示词：

```

1  def build_code_generation_prompt(task_name: str, definition: List[str], examples:
    List[Dict[str, Any]]) -> str:
2      demo_examples = examples[:10]
3      examples_text = "\n".join(f"输入: {ex['input']}\n输出: {ex['output']}" for ex
    in demo_examples)
4      return f"""
5      你是 Python 算法工程师。
6      请根据任务定义和样例生成可执行 Python 代码。
7
8      任务名: {task_name}
9      任务定义:
10     {chr(10).join(definition)}
11
12     样例:
13     {examples_text}
14
15     严格要求:
16     1. 定义函数 solve(input_text: str) -> str
17     2. input_text 是完整字符串, 例如 "[1,2,3]"
18     3. 可以使用 import 或标准库
19     4. 用 ast.literal_eval 解析输入
20     5. 返回值必须是字符串
21     6. 生成代码必须能被 exec 直接执行
22     """

```

该提示词包含五类信息：

信息类型	作用
角色设定	让模型以 Python 算法工程师身份生成代码
任务名称	告诉模型当前任务主题
任务定义	给出自然语言规则
输入输出样例	提供 few-shot 归纳依据
严格要求	限定函数签名、解析方式、返回类型和可执行性

对于 Concat Strings 任务，模型只需要从样例中归纳出“对字符串列表进行拼接”这一规则即可。前 10 个样例通常足够让模型生成正确代码。

不过，这里有一个细节问题：提示词第 2 条写的是：

```
1 input_text 是完整字符串，例如 "[1,2,3]"
```

这个例子来自数字列表任务，更适合 Collatz 或 Closest Integers 等任务。对于本任务，建议改成字符串列表示例：

```
1 input_text 是完整字符串，例如 "['p', 'that.', 'o']"
```

虽然 `ast.literal_eval` 同时适用于数字列表和字符串列表，但提示词示例与当前任务越一致，模型越不容易生成错误逻辑。

5.5 动态编译函数 `compile_solver`

代码通过 `exec` 执行模型生成的 Python 代码：

```
1 def compile_solver(code: str) -> Callable[[str], str]:
2     namespace: Dict[str, Any] = {}
3     exec(code, namespace)
4     if "solve" not in namespace:
5         raise ValueError("生成代码中未定义 solve")
6     return namespace["solve"]
```

该函数的作用是将字符串形式的 Python 代码转换成可调用函数对象。它要求生成代码中必须定义：

```
1 def solve(input_text: str) -> str:
2     ...
```

如果没有 `solve`，则直接抛出异常。该异常会被 `generate_valid_solver` 捕获，并作为错误反馈的一部分用于下一轮生成。

需要注意的是，当前实现中 `exec(code, namespace)` 没有限制可用内置函数、导入模块或文件系统访问，因此从安全角度看，它只适合在受控比赛环境或可信模型输出场景下使用。如果在开放环境中执行来自模型或用户的任意代码，应当使用沙箱、进程隔离、资源限制或白名单机制。

5.6 验证函数 `validate_solver`

验证函数用于在带标签样例上评估 solver：

```
1 def validate_solver(solve_func, examples, limit=100):
2     total = min(limit, len(examples))
3     errors = []
4     correct = 0
5     for sample in examples[:total]:
6         try:
7             pred = str(solve_func(sample["input"])).strip()
8         except Exception as e:
9             pred = f"<runtime_error: {e}>"
```

```

10         gt = str(sample["output"][0]).strip()
11         if pred == gt:
12             correct += 1
13         else:
14             errors.append({
15                 "id": sample.get("id", ""),
16                 "input": sample["input"],
17                 "gt": gt,
18                 "pred": pred
19             })
20     return correct / total if total else 0.0, errors

```

其评估标准是严格字符串匹配：

```

1 | pred == gt

```

该标准适合本任务，因为输出是一个确定字符串，不存在多个等价格格式。对于 Concat Strings，输出中任何多余空格、引号、列表符号或换行都会导致判错。

函数还捕获运行时异常，并将异常信息写入预测结果：

```

1 | pred = f"<runtime_error: {e}>"

```

这样可以避免单个样例报错导致整个验证流程中断，也方便后续将错误样例反馈给模型进行修正。

5.7 自动生成与修正函数 `generate_valid_solver`

这是当前代码中最核心的自动化模块。其流程可以概括为：

1. 第一次调用模型时，使用任务定义和前 10 个样例构造初始 prompt；
2. 提取模型生成的代码；
3. 编译代码并获取 `solve` 函数；
4. 在 examples 的前若干条样例上验证准确率；
5. 如果准确率为 100%，直接返回该 solver；
6. 如果存在错误，将前 10 个错误样例构造成错误反馈 prompt；
7. 重新调用模型生成完整代码；
8. 如果多轮尝试后仍失败，则使用回退求解器。

核心代码逻辑如下：

```

1 | for attempt in range(1, max_attempts + 1):
2 |     if attempt == 1:
3 |         prompt = build_code_generation_prompt(task_name, definition, examples)
4 |     else:
5 |         error_text = "\n".join(
6 |             f"输入: {e['input']} | 正确输出: {e['gt']} | 你的输出: {e['pred']}"
7 |             for e in last_errors[:10]

```

```

8         )
9         prompt = f"""
10 你上一次生成的 solve 函数在以下样例上出错:
11 {error_text}
12 请重新生成完整 Python 代码, 并修正逻辑。
13 严格要求:
14 - 定义 solve(input_text: str) -> str
15 - 可以使用 import 或标准库
16 - 输出字符串
17 """

```

这一设计体现了一个简单但有效的自修正机制：模型不只是一次性生成答案，而是根据本地验证结果迭代修复。对于复杂任务，这种机制可以显著提升最终 solver 的正确率。

对于本任务，由于规则很简单，通常第一轮即可生成正确代码。但保留多轮重试仍然有意义，因为模型可能输出解释文字、遗漏函数签名、返回列表而非字符串，或者错误地理解其他字符串操作任务。

5.8 预测函数 `run_predictions`

该函数用于批量运行 solver，并根据 `has_label` 参数决定是否计算准确率：

```

1  def run_predictions(solve_func, samples, has_label=False):
2      results = []
3      correct = 0
4      for s in tqdm(samples, desc="Running solver"):
5          try:
6              pred = str(solve_func(s["input"])).strip()
7          except Exception as e:
8              pred = f"<runtime_error: {e}>"
9          item = {"test_sample_id": s.get("id", ""), "prediction": pred}
10         if has_label:
11             gt = str(s["output"][0]).strip()
12             item["gt"] = gt
13             item["is_correct"] = pred == gt
14             if item["is_correct"]:
15                 correct += 1
16             results.append(item)
17         output = {"total": len(samples), "details": results}
18         if has_label:
19             output["correct"] = correct
20             output["accuracy"] = correct / len(samples) if samples else 0.0
21         return output

```

当 `has_label=True` 时，输出包含：

```

1  {
2    "total": 100,
3    "correct": 100,
4    "accuracy": 1.0,
5    "details": [
6      {
7        "test_sample_id": "...",
8        "prediction": "...",
9        "gt": "...",
10       "is_correct": true
11     }
12   ]
13 }

```

当 `has_label=False` 时，输出只包含预测结果：

```

1  {
2    "total": 100,
3    "details": [
4      {
5        "test_sample_id": "...",
6        "prediction": "..."
7      }
8    ]
9  }

```

该函数同时用于 `examples` 和 `test_samples`，因此保证了训练集评估和测试集预测使用同一套运行逻辑。

6. 主流程说明

主流程位于：

```

1  if __name__ == "__main__":

```

执行步骤如下：

6.1 配置输入与输出路径

当前代码中路径为 Windows 本地绝对路径：

```

1  file_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
Annotation\data\openseek-4_conala_concat_strings.json"
2  out_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
Annotation\rgs_q4\experiment\openseek4_collatz_results.json"
3  test_jsonl_path = r"D:\work_files\python_project\flagOS赛题三\LongContext-ICL-
Annotation\experiment\openseek-4-v1.jsonl"

```

这里存在一个命名问题：`out_path` 文件名包含 `collatz`，但当前任务是 `Conala Concat Strings`。这不会影响代码运行，但会造成实验记录混乱。建议将其修改为：

```
1 out_path = r"...\\openseek4_concat_strings_results.json"
```

6.2 加载任务数据

```
1 task_id, task_name, definition, examples, test_samples = load_task(file_path)
```

随后打印任务信息和样例数量，便于确认数据是否正确读取。

6.3 生成有效 solver

```
1 generated_code, solve_func, preview_acc, preview_errors = generate_valid_solver(  
2     task_name, definition, examples, max_attempts=3  
3 )
```

该步骤会调用模型生成代码，并在 `examples` 的前最多 200 条样例上预验证。若准确率达到 100%，则进入后续预测；否则最多重试 3 次。

6.4 评估完整 examples

```
1 example_eval = run_predictions(solve_func, examples, has_label=True)
```

该步骤用于获得完整训练/示例集上的准确率。对于本任务，正确 solver 应达到 100%。

6.5 预测 test_samples

```
1 test_eval = run_predictions(solve_func, test_samples, has_label=False)
```

该步骤生成测试集预测结果。由于测试集通常没有标准答案，因此不计算准确率。

6.6 保存完整 JSON 结果

```
1 output_data = {  
2     "task_id": task_id,  
3     "task_name": task_name,  
4     "evaluate_time": time.strftime("%Y-%m-%d %H:%M:%S", time.localtime()),  
5     "generated_code": generated_code,  
6     "preview_accuracy_on_examples": preview_acc,  
7     "preview_errors": preview_errors[:20],  
8     "examples_eval": example_eval,  
9     "test_eval": test_eval  
10 }
```

该 JSON 文件用于保存完整实验记录，包括：

- 任务信息；
- 评估时间；
- 模型生成的 solver 代码；
- 预验证准确率；
- 预验证错误样例；
- examples 完整评估结果；
- test_samples 预测结果。

这类文件适合用于调试、复盘和报告撰写。

6.7 保存 JSONL 提交文件

```
1 with open(test_jsonl_path, 'w', encoding='utf-8') as f:
2     for item in test_eval["details"]:
3         line = json.dumps(item, ensure_ascii=False)
4         f.write(line + '\n')
```

最终提交文件采用 JSONL 格式，即每一行是一个独立 JSON 对象。例如：

```
1 {"test_sample_id": "openseek-4-785d9cb1e1274411aafb19181ff05596", "prediction":
  "fkbuttonedearefIasW"}
```

JSONL 的优点是便于按行读取、增量写入和大规模评测。

7. 正确 solver 的逻辑分析

对于 Conala Concat Strings，正确解法应满足以下逻辑：

1. 去除输入字符串前后空白；
2. 使用 `ast.literal_eval` 将输入解析为 Python 列表；
3. 确认列表元素为字符串；
4. 使用 `''.join(strings)` 按原顺序拼接；
5. 返回拼接结果字符串。

最简正确实现如下：

```
1 import ast
2
3 def solve(input_text: str) -> str:
4     strings = ast.literal_eval(input_text.strip())
5     return ''.join(strings)
```

该实现的时间复杂度为：

```
1 | O(n + L)
```

其中：

- `n` 是列表元素数量；
- `L` 是所有字符串总长度。

空间复杂度为：

```
1 | O(L)
```

因为最终输出字符串需要存储所有拼接后的字符。

对于本任务，不需要排序、过滤、去重、插入分隔符或进行大小写转换。任何额外操作都可能导致答案错误。

8. 当前代码与原报告的一致性检查

现有 PDF 报告中将回退求解器描述为：

```
1 | def solve(input_text: str) -> str:
2 |     strings = ast.literal_eval(input_text.strip())
3 |     return ''.join(strings)
```

这个描述是符合 Conala Concat Strings 任务的。

但是，当前上传的 `submit.py` 中，`fallback_solver_code()` 实际内容为：

```
1 | def solve(input_text: str) -> str:
2 |     nums = ast.literal_eval(input_text.strip())
3 |     result = []
4 |     for x in nums:
5 |         if x % 2 == 0:
6 |             result.append(x // 2)
7 |         else:
8 |             result.append(x * 3 + 1)
9 |     return str(result)
```

该逻辑是 Collatz Conjecture 任务中的一步变换逻辑，并不是字符串拼接逻辑。对于本任务，如果模型生成代码失败、编译失败或验证多次不通过，程序会回退到这个错误 solver，导致最终预测完全不符合 Concat Strings 任务要求。

因此，当前代码存在一个明显问题：回退求解器未从前一个任务中正确替换为本任务逻辑。

建议修正为：

```

1 def fallback_solver_code() -> str:
2     return ''
3 import ast
4
5 def solve(input_text: str) -> str:
6     strings = ast.literal_eval(input_text.strip())
7     return ''.join(strings)
8     ''.strip()

```

如果希望进一步增强鲁棒性，可以加入类型转换：

```

1 def fallback_solver_code() -> str:
2     return ''
3 import ast
4
5 def solve(input_text: str) -> str:
6     items = ast.literal_eval(input_text.strip())
7     return ''.join(str(x) for x in items)
8     ''.strip()

```

不过，如果数据集明确保证元素均为字符串，则第一种实现更严格，也更符合任务定义。

9. 当前代码存在的问题与修改建议

9.1 回退求解器逻辑错误

问题：当前 `fallback_solver_code()` 是 Collatz 逻辑，不适用于 Concat Strings。

影响：当模型生成失败或验证失败时，程序会使用错误 solver，导致输出为数字列表变换结果，而不是字符串拼接结果。

修改建议：改为：

```

1 def fallback_solver_code() -> str:
2     return ''
3 import ast
4
5 def solve(input_text: str) -> str:
6     strings = ast.literal_eval(input_text.strip())
7     return ''.join(strings)
8     ''.strip()

```

9.2 输出文件命名不一致

问题：`out_path` 使用了 `openseek4_collatz_results.json`，但当前任务是 Concat Strings。

影响：不会影响程序运行，但会影响实验文件管理，容易和 OpenSeek-3 Collatz Conjecture 任务混淆。

修改建议：


```
1 out_path = r"...\\openseek4_concat_strings_results.json"
```

9.3 Prompt 示例与任务不完全一致

问题：提示词中写道：

```
1 input_text 是完整字符串，例如 "[1,2,3]"
```

这更像数字列表任务示例。对于本任务，更建议改成：

```
1 input_text 是完整字符串，例如 "['p', 'that.', 'o']"
```

影响：虽然不一定导致错误，但可能增加模型误解任务类型的概率。

9.4 API 配置硬编码

问题：API 地址和模型名称写在代码中，不利于跨机器运行，也不利于提交版本管理。

修改建议：使用环境变量：

```
1 client = OpenAI(  
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),  
3     base_url=os.getenv("OPENAI_BASE_URL"),  
4 )  
5  
6 MODEL_NAME = os.getenv("MODEL_NAME", "/Qwen3-4B/Qwen/Qwen3-4B")
```

9.5 `exec` 缺少安全限制

问题：`compile_solver` 直接执行模型生成代码，存在安全风险。

影响：在受控比赛环境中可以接受；在开放环境中不建议直接执行未知代码。

修改建议：

- 使用独立子进程运行生成代码；
- 设置超时；
- 限制文件系统访问；
- 限制可导入模块；
- 对 AST 做静态检查，只允许函数定义、导入标准库和简单表达式。

9.6 缺少运行超时控制

问题：如果模型生成了死循环代码，`validate_solver` 或 `run_predictions` 可能卡住。

修改建议：可以使用 `multiprocessing` 或 `signal` 添加单样例超时控制。对于本任务，正确逻辑非常简单，但作为通用自动代码生成框架，超时控制仍然重要。

10. 长上下文与提示词规模说明

当前代码的提示词由以下部分组成：

1. 任务角色说明；
2. 任务名称；
3. 任务定义；
4. examples 前 10 条样例；
5. 函数签名和输出格式要求。

对应代码为：

```
1 demo_examples = examples[:10]
```

因此，当前代码并没有构造超长 prompt，也没有显式保证 prompt 超过 20k tokens 或 20k 字符。如果比赛要求必须体现长上下文能力，当前版本需要进一步修改。例如：

```
1 def build_long_context_prompt(task_name, definition, examples, min_chars=20000):
2     blocks = []
3     blocks.append("你是 Python 算法工程师。请根据任务定义和大量样例生成 solve 函数。")
4     blocks.append(f"任务名: {task_name}")
5     blocks.append("任务定义: \n" + "\n".join(definition))
6     blocks.append("样例: ")
7
8     for ex in examples:
9         blocks.append(f"输入: {ex['input']}\n输出: {ex['output']}\n")
10        if len("\n".join(blocks)) >= min_chars:
11            break
12
13        blocks.append("""
14严格要求：
151. 定义 solve(input_text: str) -> str
162. input_text 是完整字符串，例如 "['p', 'that.', 'o']"
173. 使用 ast.literal_eval 解析输入
184. 返回拼接后的字符串
195. 只输出完整 Python 代码
20""")
21    return "\n\n".join(blocks)
```

不过，对于 Concat Strings 本身，超长 prompt 并不是算法正确性的必要条件。由于规则简单，少量样例已经足够归纳出 `''.join(strings)`。如果只是追求解题效果，当前 few-shot prompt 更简洁；如果需要满足长上下文实验要求，则应当明确增加样例数量或加入任务分析知识块。